# Actividad Integradora 3
# Sistema web dinámico en PHP con MariaDB y sentencias preparadas

**Facultad:** Facultad de Ingenierías
**Carrera:** Sistemas Inteligentes
**Asignatura:** Programación de Sistemas Web

**Integrante:**
**José Esteban Dávalos Yépez**

**Repositorio de GitHub:**
https://github.com/jedy-knight11/AI3_DisenoWeb_JoseDavalos

**Fecha de entrega: 8 de noviembre de 2026**

## Sección 1: Justificación técnica

### 1.1 Análisis comparativo entre MySQLi y PDO

PHP ofrece dos extensiones para trabajar con MySQL y MariaDB. MySQLi (MySQL improved) está diseñada exclusivamente para esos motores. PDO (PHP Data Objects) es una capa de abstracción que define una misma interfaz para distintos motores, cada uno con su propio controlador (pdo_mysql, pdo_pgsql, pdo_sqlite, pdo_sqlsrv, entre otros).

| **Criterio** | **MySQLi** | **PDO** |
|---|---|---|
| Arquitectura | Extensión específica de MySQL/MariaDB. Interfaz procedimental y orientada a objetos. | Capa de abstracción con interfaz orientada a objetos y controladores por motor. |
| Portabilidad | Solo MySQL y MariaDB. | Más de diez motores con la misma API; cambiar de motor implica sobre todo cambiar el DSN. |
| Sentencias preparadas | Sí, solo con marcadores posicionales (`?`) y `bind_param()` con cadena de tipos. | Sí, con marcadores posicionales o con nombre (`:correo`), más legibles. |
| Manejo de errores | Desde PHP 8.1 lanza `mysqli_sql_exception` por defecto. | Modo configurable; con `ERRMODE_EXCEPTION` lanza `PDOException` con el código SQLSTATE y el código del motor. |
| Rendimiento | Ligeramente más rápido en pruebas aisladas por no tener capa de abstracción. | Diferencia mínima en aplicaciones reales; el tiempo lo domina la consulta en el motor. |
| Seguridad | Segura si se usan sentencias preparadas. | Igual de segura; con `ATTR_EMULATE_PREPARES = false` la preparación ocurre en el servidor. |
| Funciones exclusivas | Consultas asíncronas y múltiples (`multi_query`) propias de MySQL. | Solo las comunes a todos los motores. |

**Selección: PDO.** Se eligió por tres razones:

- **Portabilidad:** el módulo podría migrar a PostgreSQL o SQLite cambiando principalmente la cadena de conexión.
- **Abstracción:** los parámetros con nombre hacen las consultas más legibles y la misma API sirve para cualquier proyecto futuro.
- **Manejo de excepciones:** `PDOException` entrega el código del error, lo que permite distinguir un correo duplicado (código 1062) de una falla del servidor y mostrar a cada caso un mensaje distinto.

Las funciones exclusivas de MySQLi no son necesarias para este módulo y la diferencia de rendimiento no es significativa.

### 1.2 Medidas de seguridad implementadas

- **Inyección SQL:** todas las consultas usan `prepare()`, `bindValue()` y `execute()`. Con la emulación desactivada, el SQL y los datos viajan por separado al motor, que nunca interpreta un dato como código.
- **XSS:** toda salida pasa por la función `e()`, que aplica `htmlspecialchars()` con `ENT_QUOTES` y UTF-8. Un `<script>` guardado se muestra como texto.
- **Validación en dos capas:** HTML5 (`required`, `pattern`, `type="email"`) para el usuario, y `ValidadorCliente` en el servidor, porque una petición puede enviarse sin pasar por el formulario.
- **Credenciales y errores:** la conexión vive en `conexion.php` dentro de un `try-catch`. Si falla, el detalle se registra en el log y el usuario solo ve un mensaje genérico. Una excepción sin capturar podría mostrar en su traza los argumentos del constructor de PDO, incluido el usuario y la contraseña. Las credenciales están en `.env` o `config/config.php`, ambos en `.gitignore`.
- **Otras medidas:** token CSRF en el formulario, patrón Post/Redirect/Get, clave única en el correo, usuario de base de datos sin privilegios de administrador y solo la carpeta `public/` expuesta en el servidor web.

```
$pdo = new PDO($dsn, $usuario, $clave, [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
]);
```

Fragmento de config/conexion.php: errores como excepciones y preparación real en el servidor.

## Sección 2: Arquitectura del sistema

El sistema usa una arquitectura modular inspirada en MVC. Cada carpeta tiene una responsabilidad:

- **Modelo** (`src/ClienteRepositorio.php`): única clase que contiene SQL.
- **Validación** (`src/ValidadorCliente.php`): reglas de negocio de los datos recibidos.
- **Controladores** (`public/guardar.php`, `public/listado.php`): reciben la petición y coordinan los módulos.
- **Vistas** (`public/index.php` y `views/`): generan el HTML.
- **Configuración** (`config/conexion.php`): crea una única conexión PDO por petición.

### 2.1 Flujo de registro (Create)

1. Navegador: el usuario completa el formulario; HTML5 valida antes de enviar

↓ POST con los datos y el token CSRF

2. guardar.php: verifica el token y llama a ValidadorCliente (validación del servidor)

↓ datos limpios

3. ClienteRepositorio::crear(): prepare(), bindValue() y execute()

↓ SQL y datos por separado

4. MariaDB: inserta en clientes; aplica la clave foránea y el correo único

↓ respuesta 303 (Post/Redirect/Get)

5. Navegador: es redirigido a listado.php con un mensaje de éxito

Si la validación falla o el correo ya existe, `guardar.php` redirige de vuelta al formulario con los errores y los datos ya escritos, guardados en la sesión.

### 2.2 Flujo de consulta (Read)

1. Navegador: GET a listado.php

↓

2. ClienteRepositorio::listar(): consulta preparada con JOIN entre clientes y servicios

↓ filas como arreglos asociativos

3. listado.php: imprime cada dato a través de e() (htmlspecialchars) en una tabla responsiva

### 2.3 Diseño de la base de datos

La base **db_sistema_web** tiene dos tablas. **servicios** es un catálogo; **clientes** es la tabla principal, con nueve campos: `id` (clave primaria autoincremental), `nombres`, `correo` (único), `telefono`, `empresa`, `servicio_id` (clave foránea), `mensaje` y los campos de auditoría `fecha_creacion` y `fecha_actualizacion`. El nombre del servicio no se repite en cada cliente: se guarda una sola vez en el catálogo y se relaciona por su id (tercera forma normal). El script DDL está en `database/db_sistema_web.sql`.

El entorno local se levanta con Docker Compose (MariaDB 11.4, PHP 8.3 con Apache y phpMyAdmin). El README también explica cómo desplegarlo con XAMPP o Laragon.

## Sección 3: Evidencias de ejecución

### 3.1 Estructura de la tabla en phpMyAdmin

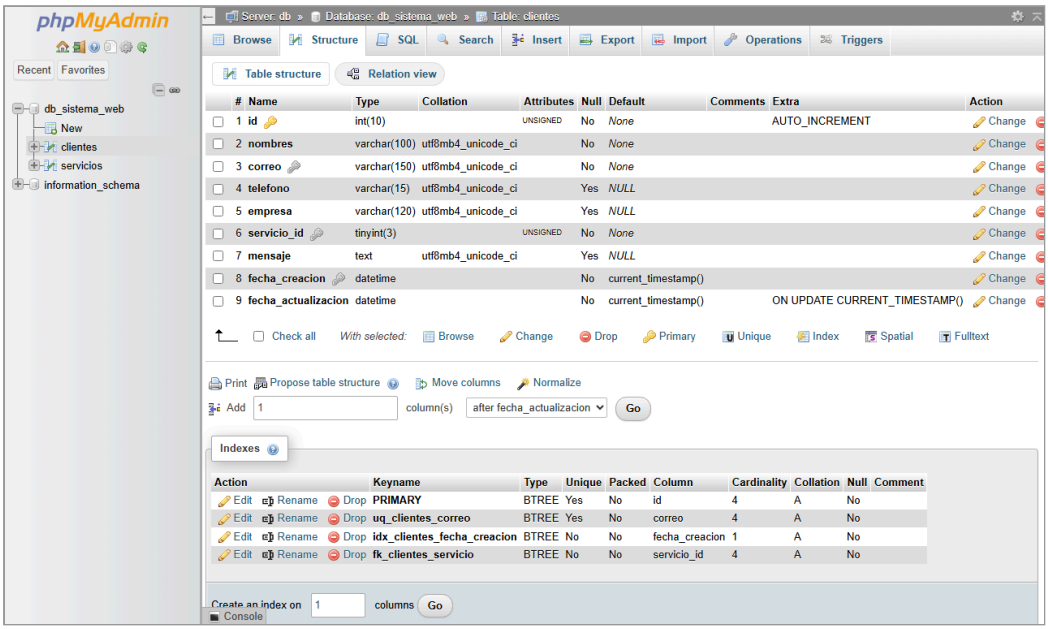



Figura 1. Estructura de la tabla clientes en phpMyAdmin

**Interpretación:** la tabla tiene nueve campos con tipos acordes a su contenido: `id` es INT sin signo con AUTO_INCREMENT; los textos usan VARCHAR con longitud máxima o TEXT; los campos opcionales aceptan NULL. Las fechas toman `current_timestamp()` por defecto, y `fecha_actualizacion` se renueva sola con ON UPDATE. En los índices se ven la clave primaria, el índice único del correo y la clave foránea hacia servicios.

### 3.2 Código con sentencias preparadas

**ClienteRepositorio.php** · src/ClienteRepositorio.php

```php

    /**
     * Inserta un cliente ya validado y devuelve su id.
     * Los valores viajan separados del SQL, así que una comilla o un ";" en un campo
     * se guardan como texto y nunca se interpretan como parte de la consulta.
     *
     * @param array<string, string> $datos
     */
    public function crear(array $datos): int
    {
        $sql = 'INSERT INTO clientes (nombres, correo, telefono, empresa, servicio_id, mensaje)
                VALUES (:nombres, :correo, :telefono, :empresa, :servicio_id, :mensaje)';

        $stmt = $this->pdo->prepare($sql);
        $stmt->bindValue(':nombres', $datos['nombres'], PDO::PARAM_STR);
        $stmt->bindValue(':correo', $datos['correo'], PDO::PARAM_STR);
        $this->vincularOpcional($stmt, ':telefono', $datos['telefono']);
        $this->vincularOpcional($stmt, ':empresa', $datos['empresa']);
        $stmt->bindValue(':servicio_id', (int) $datos['servicio_id'], PDO::PARAM_INT);
        $this->vincularOpcional($stmt, ':mensaje', $datos['mensaje']);
        $stmt->execute();

        return (int) $this->pdo->lastInsertId();
    }
```

Figura 2. Método crear() de src/ClienteRepositorio.php

**Interpretación:** la consulta tiene marcadores con nombre (`:nombres`, `:correo`...) en lugar de los valores. `prepare()` envía la estructura al motor; `bindValue()` asocia cada valor con su tipo (`PARAM_STR`, `PARAM_INT` o `PARAM_NULL` para campos vacíos), y `execute()` ejecuta. Como los datos nunca se concatenan al SQL, una comilla o un punto y coma escritos en el formulario no pueden alterar la consulta.

### 3.3 Formulario en ejecución con datos de prueba

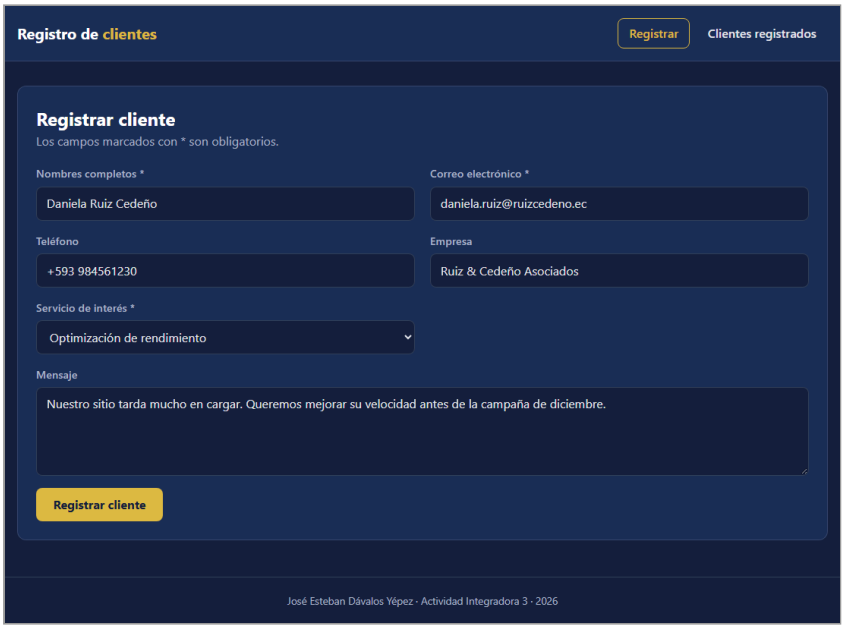



Figura 3. Formulario de registro con datos de prueba

**Interpretación:** el formulario se ejecuta en `http://localhost:8080`. Los campos con asterisco son obligatorios. El navegador bloquea el envío de campos vacíos o con formato inválido, y el servidor repite esas validaciones. La lista de servicios se carga desde la base de datos. Estos datos se enviaron y aparecen como registro 5 en la figura 4.

### 3.4 Tabla web con los registros insertados

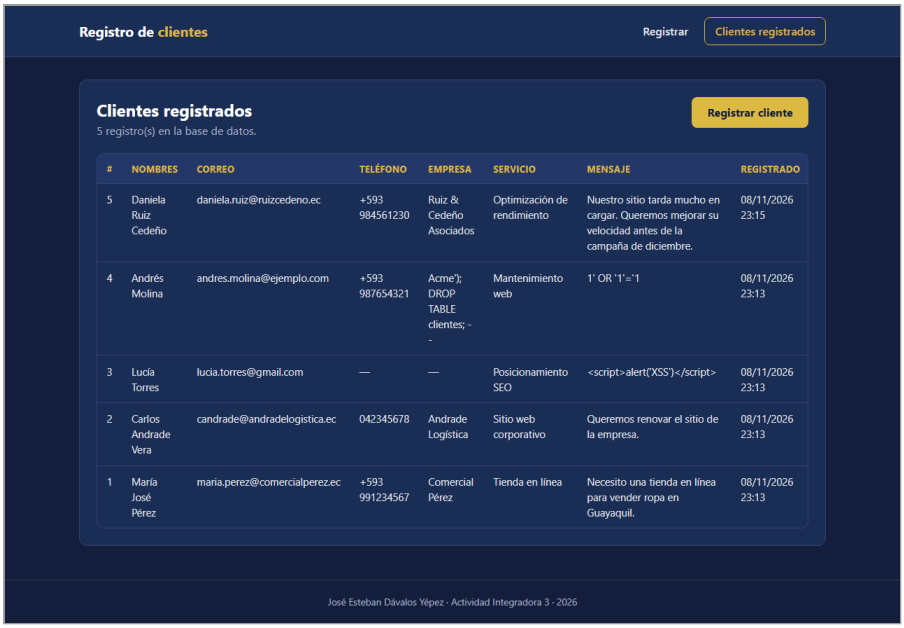



Figura 4. Clientes registrados, consultados desde la base de datos

**Interpretación:** la tabla muestra los cinco registros, del más reciente al más antiguo, con el nombre del servicio obtenido mediante un JOIN. Dos registros son pruebas de seguridad:

- **Registro 4:** contiene `Acme'); DROP TABLE clientes; --` y `1' OR '1'='1`. Se guardaron como texto y la tabla sigue existiendo, lo que demuestra que las sentencias preparadas impidieron la inyección SQL.
- **Registro 3:** contiene `<script>alert('XSS')</script>`. Se muestra como texto y no se ejecuta, gracias a `htmlspecialchars()`.

En celular, la tabla se transforma en tarjetas para no requerir desplazamiento horizontal.

### 3.5 Historial de commits

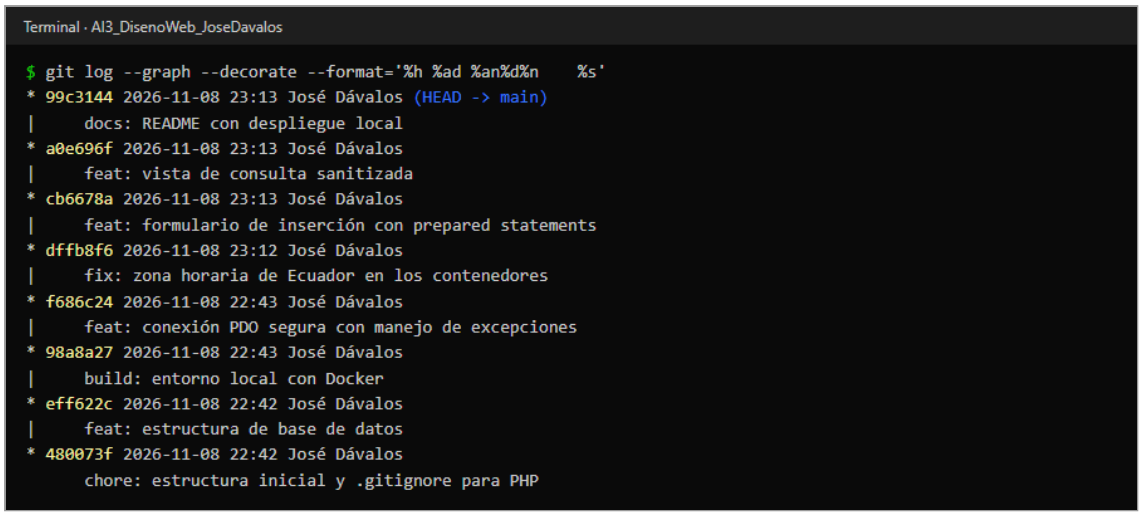



Figura 5. Historial de commits (git log)

**Interpretación:** cada commit agrupa un único cambio con sentido propio y sigue la convención de prefijos (`chore`, `feat`, `build`, `fix`, `docs`). El historial muestra el orden de construcción: base de datos, entorno, conexión, inserción y consulta. El commit `fix` corrige un problema detectado en las pruebas: MariaDB guardaba las fechas de auditoría en UTC, cinco horas adelantadas respecto a Ecuador.

## Sección 4: Enlaces del proyecto

Repositorio público de GitHub:
https://github.com/jedy-knight11/AI3_DisenoWeb_JoseDavalos

Las instrucciones de despliegue local están en el archivo README.md del repositorio.

## Declaración de uso de inteligencia artificial

Utilicé Claude (Anthropic), mediante la aplicación Claude Code, para analizar el enunciado, generar el código del módulo, configurar el entorno con Docker, ejecutar las pruebas de seguridad, tomar las capturas y redactar este informe. Yo aporté el enunciado y los datos de la portada, y aprobé las decisiones del proyecto.